

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт по лабораторной работе № 8
«Графовые алгоритмы DFS, BFS, Дейкстра»

Выполнил работу
Волков Владимир Алексеевич
Академическая группа
J3112
Принято
Практик, Максим Дунаев

Санкт-Петербург

Структура отчёта:

1. Верхняя часть кода

```
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <stack>
5  #include <cmath>
6  #include <unordered_map>
7  #include <limits>
8  #include <algorithm>
9  #include <functional>
10 #include <ctime>
11 #include <fstream>
12 #include <sstream>
13
14 using namespace std;
15
16 //Узел - это пара координат (долгота, широта).
17 using Node = pair<double, double>;
18
19 //Хэш-функция для узла
20 struct NodeHash {
21     size_t operator()(const Node& node) const {
22         auto hash1 = hash<double>{}(node.first);
23         auto hash2 = hash<double>{}(node.second);
24         return hash1 ^ (hash2 << 1);
25     }
26 };
27
28 struct Graph {
29     unordered_map<Node, vector<pair<Node, double>>, NodeHash> adj_list;
30
31     // Добавляет ребро в граф между двумя узлами с указанным весом.
32     void add_edge(double lon1, double lat1, double lon2, double lat2, double weight) {
33         adj_list[{lon1, lat1}].push_back({lon2, lat2}, weight);
34         adj_list[{lon2, lat2}].push_back({lon1, lat1}, weight);
35     }
36
37     // Читает данные из файла и заполняет граф.
38     // Время:  $O(L + E)$ , где  $L$  – количество строк,  $E$  – количество рёбер.
39     // Память:  $O(V + E)$ , где  $V$  – количество узлов,  $E$  – количество рёбер.
40     void get_file_data(const string& filename) {
41         ifstream file(filename);
42         if (!file.is_open()) {
43             cerr << "Error: Could not open file " << filename << endl;
44             return;
45         }
46     }
```

```

47     string line;
48     while (getline(file, line)) {
49         size_t pos = line.find(':');
50         if (pos == string::npos) {
51             cerr << "Invalid line format: " << line << endl;
52             continue;
53         }
54
55         // Разбор координат узлов
56         string source_coords = line.substr(0, pos);
57         stringstream source_stream(source_coords);
58         double lon1, lat1;
59         char comma;
60         source_stream >> lon1 >> comma >> lat1;
61
62         // Разбор рёбер
63         string edges = line.substr(pos + 1);
64         stringstream edge_stream(edges);
65         string edge;
66         while (getline(edge_stream, edge, ';')) {
67             stringstream edge_data(edge);
68             double lon2, lat2, weight;
69             edge_data >> lon2 >> comma >> lat2 >> comma >> weight;
70
71             // Добавляем ребро в граф
72             add_edge(lon1, lat1, lon2, lat2, weight);
73         }
74     }
75
76     file.close();
77 }

```

```

79 // Находит ближайший узел в графе к заданным координатам.
80 // Время: O(V), где V – количество узлов.
81 // Память: O(1).
82 Node find_nearest_node(double lon, double lat) {
83     Node closest;
84     double min_distance = numeric_limits<double>::max();
85
86     for (const auto& node : adj_list) {
87         double distance = sqrt(pow(node.first.first - lon, 2) + pow(node.first.second - lat, 2));
88         if (distance < min_distance) {
89             min_distance = distance;
90             closest = node.first;
91         }
92     }
93     return closest;
94 }
95
96 // Выводит путь в читаемом формате.
97 // Время: O(P), где P – длина пути.
98 // Память: O(1).
99 void print_path(const vector<Node>& path) {
100     if (path.empty()) {
101         cout << "Путь не найден" << endl;
102         return;
103     }
104     for (size_t i = 0; i < path.size(); ++i) {
105         cout << "(" << path[i].first << ", " << path[i].second << ")";
106         if (i != path.size() - 1) cout << " -> ";
107     }
108     cout << endl;
109 }
110 };
111

```

Узел и его хэш-функция:

Double – долгота и широта

Тип данных `pair<double, double>` используется для удобства хранения двух значений, относящихся к одному узлу.

Хэш-функция для узла `Node`. Она позволяет использовать узлы `Node` в качестве ключей для контейнеров `unordered_map`.

- `node.first` — это долгота узла.

- `node.second` — это широта узла.

- Стандартная хэш-функция `hash<double>{}` вычисляет два хэша:

`hash1` для долготы (`node.first`);

`hash2` для широты (`node.second`).

Эти два хэша комбинируются с помощью операции XOR (^) и побитового сдвига (<< 1), чтобы избежать коллизий.

Структура Graph:

Граф представлен в виде списка смежности (`adj_list`), где:

Ключ: `Node` (узел графа, пара координат);

Значение: `vector<pair<Node, double>>` (список соседних узлов с указанием веса рёбер между ними).

Для вычисления хэша узлов используется структура `NodeHash`.

Метод `add_edge`:

Добавляет двустороннее (неориентированное) ребро между двумя узлами графа.

Используется список смежности (`adj_list`) для хранения узлов и их связей.

Узел с координатами (`lon1, lat1`) соединяется с узлом (`lon2, lat2`), и связь имеет вес `weight`.
В обоих направлениях добавляются записи:

`(lon1, lat1) → (lon2, lat2);`

`(lon2, lat2) → (lon1, lat1).`

Метод `get_file_data`:

Читает данные о графе из файла и заполняет список смежности (`adj_list`)

Открывает файл с заданным именем `filename`.

Считывает строки файла:

- Формат строки: долгота,широта;долгота,широта,вес;долгота,широта,вес;....
- Пример: 30.0,60.0;30.1,60.1,1.0;30.2,60.2,2.0.

Разделяет строку на:

- Координаты исходного узла (`source_coords`).

- Рёбра (edges) к другим узлам.

Парсит координаты и вес каждого ребра и добавляет их в граф с помощью `add_edge`.

Метод `find_nearest_node`

Находит ближайший узел графа к заданным координатам.

1. Инициализирует `min_distance` бесконечно большим значением.
2. Для каждого узла графа вычисляет Евклидово расстояние до заданной точки:
 - Расстояние: $\sqrt{(x1-x2)^2+(y1-y2)^2}$
3. Если расстояние меньше текущего `min_distance`, обновляет ближайший узел (`closest`) и его расстояние.

Метод `print_path`

Выводит путь (список узлов) в удобном для чтения формате.

Если путь пустой, сообщает, что путь не найден.

Перебирает узлы пути:

- Выводит координаты каждого узла.
- Разделяет узлы стрелками `->` для читаемости.

2. Алгоритм кода BFS

```

112 // BFS: Поиск в ширину
113 // Находит кратчайший путь в графе без учёта весов рёбер.
114 // Время:  $O(V + E)$ , где  $V$  – количество узлов,  $E$  – количество рёбер.
115 // Память:  $O(V)$ .
116 vector<Node> bfs(Graph& graph, Node start, Node end) {
117     queue<Node> to_visit;
118     unordered_map<Node, Node, NodeHash> parent;
119     unordered_map<Node, bool, NodeHash> visited;
120
121     to_visit.push(start);
122     visited[start] = true;
123     parent[start] = start;
124
125     while (!to_visit.empty()) {
126         Node current = to_visit.front();
127         to_visit.pop();
128
129         if (current == end) {
130             vector<Node> path;
131             for (Node at = end; at != start; at = parent[at]) {
132                 path.push_back(at);
133             }
134             path.push_back(start);
135             reverse(path.begin(), path.end());
136             return path;
137         }
138
139         for (const auto& neighbor : graph.adj_list[current]) {
140             if (!visited[neighbor.first]) {
141                 visited[neighbor.first] = true;
142                 parent[neighbor.first] = current;
143                 to_visit.push(neighbor.first);
144             }
145         }
146     }
147
148     return {}; // Путь не найден
149 }

```

Очередь to_visit:

- FIFO-структура (первым пришёл — первым вышел), которая хранит узлы, которые нужно посетить.
- Узлы добавляются в порядке их обнаружения, обеспечивая проход по уровням графа.

unordered_map<Node, Node, NodeHash> parent:

- Словарь, который хранит информацию о том, из какого узла мы пришли в текущий узел.
- Используется для восстановления пути от стартового узла к целевому.

unordered_map<Node, bool, NodeHash> visited:

- Словарь, чтобы отслеживать, какие узлы мы уже посетили.
- Это предотвращает повторное посещение одного и того же узла, избегая бесконечных циклов.

Стартовый узел `start` добавляется в очередь `to_visit`, помечается как посещённый в словаре `visited` и сам становится своим родителем в словаре `parent`.

Это необходимо, чтобы алгоритм начал работу с узла `start`.

Пока очередь `to_visit` не пуста, продолжается обработка узлов графа.

Извлекаем первый узел `current` из очереди. Это узел, который нужно обработать на текущем шаге.

Удаляем этот узел из очереди.

Если текущий узел `current` совпадает с целевым узлом `end`, то путь найден.

Восстанавливаем путь с помощью словаря `parent`:

1. Начинаем с целевого узла `end`.
2. Переходим к родителю каждого узла, пока не дойдём до стартового узла `start`.
3. Добавляем узлы в вектор `path` в обратном порядке.
4. Разворачиваем вектор `path` с помощью `reverse`, чтобы получить путь от `start` к `end`.

Возвращаем найденный путь.

Для каждого соседа `neighbor` текущего узла `current`:

- 1) Проверяем, посещали ли этот сосед с помощью `visited`.
- 2) Если сосед ещё не посещён:
 - Помечаем его как посещённый (`visited[neighbor.first] = true`).
 - Записываем в `parent`, что мы пришли в этот узел из `current`.
 - Добавляем соседа в очередь `to_visit` для дальнейшей обработки.

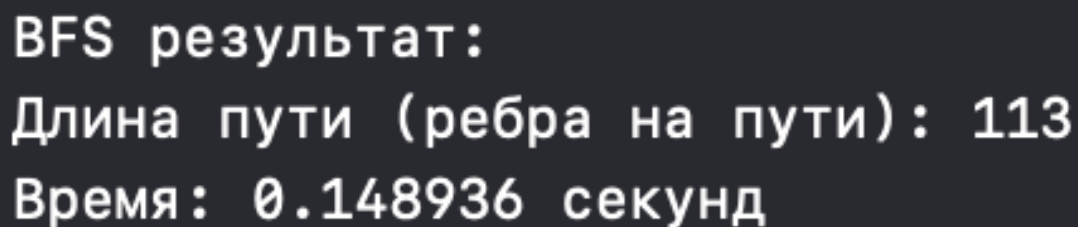
Если очередь пуста, а целевой узел `end` не был найден, значит, пути между `start` и `end` не существует.

Возвращается пустой вектор `{}`, сигнализирующий об отсутствии пути.

Общее объяснение:

1. На первом шаге он обрабатывает все узлы, которые находятся на расстоянии 1 ребра от стартового узла `start`.
2. На втором шаге обрабатывает узлы, которые находятся на расстоянии 2 рёбер, и так далее.
3. Благодаря этому свойству BFS находит кратчайший путь (по числу рёбер) в неориентированном или равномерно взвешенном графе.

3. Скрины с временем работы алгоритма BFS



BFS результат:
Длина пути (ребра на пути): 113
Время: 0.148936 секунд

4. Алгоритм DFS

```
150
151 // DFS: Поиск в глубину
152 // Находит путь между двумя узлами. Не гарантирует кратчайший путь.
153 // Время:  $O(V + E)$ , где  $V$  – количество узлов,  $E$  – количество рёбер.
154 // Память:  $O(V)$ .
155 vector<Node> dfs(Graph& graph, Node start, Node end) {
156     stack<Node> to_visit;
157     unordered_map<Node, Node, NodeHash> parent;
158     unordered_map<Node, bool, NodeHash> visited;
159
160     to_visit.push(start);
161     parent[start] = start;
162
163     while (!to_visit.empty()) {
164         Node current = to_visit.top();
165         to_visit.pop();
166
167         if (visited[current]) continue;
168         visited[current] = true;
169
170         if (current == end) {
171             vector<Node> path;
172             for (Node at = end; at != start; at = parent[at]) {
173                 path.push_back(at);
174             }
175             path.push_back(start);
176             reverse(path.begin(), path.end());
177             return path;
178         }
179
180         for (const auto& neighbor : graph.adj_list[current]) {
181             if (!visited[neighbor.first]) {
182                 parent[neighbor.first] = current;
183                 to_visit.push(neighbor.first);
184             }
185         }
186     }
187
188     return {}; // Путь не найден
189 }
```

Стек `to_visit`:

Используется для хранения узлов, которые нужно посетить.

Узлы добавляются в стек в порядке их обнаружения, но последний добавленный узел обрабатывается первым (LIFO — последний пришёл, первый вышел).

unordered_map<Node, Node, NodeHash> parent:

Хранит информацию о том, из какого узла мы пришли в текущий узел.

Используется для восстановления пути от стартового узла к целевому.

unordered_map<Node, bool, NodeHash> visited:

Хранит информацию о том, посещали ли мы уже данный узел.

Предотвращает заикливание и повторное посещение узлов.

Стартовый узел start помещается в стек to_visit.

В словарь parent добавляется запись, что узел start сам является своим родителем. Это начальная точка для восстановления пути.

Пока стек to_visit не пустой, продолжается обработка узлов.

Извлекаем верхний элемент стека to_visit и назначаем его текущим узлом current.

Если узел current уже был посещён, пропускаем его обработку и переходим к следующей итерации (continue).
Если узел ещё не был посещён:

- Помечаем его как посещённый в словаре visited.

Если текущий узел current совпадает с целевым узлом end, значит, путь найден.
Для восстановления пути используется словарь parent:

1. Начинаем с целевого узла end.
2. Проходим по родителям каждого узла, пока не дойдём до стартового узла start.
3. Добавляем узлы в вектор path в обратном порядке (от конца к началу пути).
4. Разворачиваем вектор path с помощью reverse, чтобы получить путь от start к end.

Возвращаем путь.

Для каждого соседа neighbor текущего узла current:

1. Проверяем, посещён ли этот сосед с помощью visited.
2. Если сосед ещё не посещён:

Добавляем в словарь parent информацию о том, что в этот соседний узел мы пришли из узла current.

Добавляем соседа в стек to_visit для дальнейшей обработки.

Если стек пуст, а целевой узел end не был найден, то пути от start к end в графе нет.

Возвращаем пустой вектор {}, сигнализирующий об отсутствии пути.

Общее объяснение:

- DFS исследует граф "вглубь" по одному пути, пока не найдёт целевой узел или не упрётся в тупик.

- Если встречается тупик, алгоритм возвращается к последнему посещённому узлу (сохраняемому в стеке) и продолжает поиск с его соседей.
- В отличие от BFS, который проходит граф "по уровням", DFS может не найти кратчайший путь в графе.

Стек to_visit:

- Узлы добавляются в стек, чтобы продолжить исследование пути.
- Стек обеспечивает обход "вглубь" (LIFO — последний пришёл, первый вышел).

Словарь visited:

- Используется для отслеживания посещённых узлов.
- Предотвращает повторный заход в один и тот же узел, исключая заикливания.

Словарь parent:

- Хранит родителя каждого узла, что позволяет восстановить путь от целевого узла end к стартовому start.

5. Скрин с временем работы алгоритма DFS

```
DFS результат:
Длина пути (ребра на пути): 60513
Время: 0.415601 секунд
```

6. Алгоритм Дейкстры

```

190
191 // Dijkstra: Алгоритм Дейкстры
192 // Находит кратчайший путь с учётом весов рёбер.
193 // Время:  $O((V + E) \log V)$ , где  $V$  – количество узлов,  $E$  – количество рёбер.
194 // Память:  $O(V)$ .
195 vector<Node> dijkstra(Graph& graph, Node start, Node end) {
196     unordered_map<Node, double, NodeHash> dist;
197     unordered_map<Node, Node, NodeHash> parent;
198     priority_queue<pair<double, Node>, vector<pair<double, Node>>, greater<>> pq;
199
200     for (const auto& node : graph.adj_list) {
201         dist[node.first] = numeric_limits<double>::max();
202     }
203     dist[start] = 0.0;
204     pq.push({0.0, start});
205
206     while (!pq.empty()) {
207         auto [current_dist, current] = pq.top();
208         pq.pop();
209
210         if (current == end) {
211             vector<Node> path;
212             for (Node at = end; at != start; at = parent[at]) {
213                 path.push_back(at);
214             }
215             path.push_back(start);
216             reverse(path.begin(), path.end());
217             return path;
218         }
219
220         for (const auto& neighbor : graph.adj_list[current]) {
221             double new_dist = current_dist + neighbor.second;
222             if (new_dist < dist[neighbor.first]) {
223                 dist[neighbor.first] = new_dist;
224                 parent[neighbor.first] = current;
225                 pq.push({new_dist, neighbor.first});
226             }
227         }
228     }
229
230     return {}; // Путь не найден
231 }
232

```

unordered_map<Node, double, NodeHash> dist:

Хранит минимальные расстояния от начального узла start до каждого узла графа.

Изначально для всех узлов расстояние задаётся как бесконечность, кроме начального узла start, для которого расстояние равно 0.

unordered_map<Node, Node, NodeHash> parent:

Хранит информацию о предыдущем узле на кратчайшем пути к каждому узлу.

Используется для восстановления пути от начального узла start до целевого узла end.

priority_queue<pair<double, Node>, vector<pair<double, Node>>, greater<>> pq:

Это приоритетная очередь, которая хранит узлы вместе с текущим минимальным расстоянием до них.

Узлы с меньшим расстоянием обрабатываются первыми.

Для каждого узла графа:

- Задаём начальное расстояние как бесконечность (`numeric_limits<double>::max()`).

Для стартового узла `start` расстояние устанавливается равным `0.0`, так как путь до самого себя равен нулю.

Добавляем начальный узел `start` в приоритетную очередь `pq` с расстоянием `0.0`.

Пока приоритетная очередь `pq` не пуста, продолжаем извлекать узлы с минимальным текущим расстоянием для обработки.

Извлекаем узел `current` с минимальным расстоянием `current_dist` из приоритетной очереди.

Это гарантирует, что данный узел обрабатывается первым среди всех остальных узлов, которые ещё находятся в очереди.

Если текущий узел `current` совпадает с целевым узлом `end`, то путь найден.

Для восстановления пути:

1. Начинаем с целевого узла `end`.
2. С помощью словаря `parent` проходим назад по пути до начального узла `start`.
3. Добавляем узлы в вектор `path` в обратном порядке.
4. Разворачиваем путь, чтобы он был от `start` к `end`.

Возвращаем вектор `path`, представляющий кратчайший путь.

Для каждого соседа `neighbor` текущего узла `current`:

- 1) Вычисляем потенциальное новое расстояние `new_dist` как сумму текущего расстояния `current_dist` и веса рёбра `neighbor.second`.
- 2) Если это новое расстояние меньше текущего известного расстояния `dist[neighbor.first]`:
 - Обновляем минимальное расстояние для узла `neighbor.first` в словаре `dist`.
 - Записываем в словарь `parent`, что в узел `neighbor.first` мы пришли из узла `current`.
 - Добавляем соседа `neighbor.first` в приоритетную очередь `pq` с обновлённым расстоянием `new_dist`.

Если очередь пуста, а целевой узел `end` не был найден, значит, пути от `start` к `end` не существует.

Возвращается пустой вектор `{}`, сигнализирующий об отсутствии пути.

Общее объяснение:

1. Он всегда выбирает узел с минимальным текущим расстоянием (за счёт использования приоритетной очереди).
2. После обработки узла расстояние до него считается "окончательным", так как более короткого пути до этого узла не существует.
3. Последовательно обновляет минимальные расстояния до соседних узлов, гарантируя, что приоритетная очередь всегда содержит только актуальные значения.

Приоритетная очередь:

- Узлы с меньшим расстоянием всегда извлекаются первыми.

- Это позволяет эффективно находить кратчайший путь.

Словарь `dist`:

- Гарантирует, что на каждом шаге мы храним только минимальные расстояния.

Словарь `parent`:

- Позволяет восстановить путь от целевого узла `end` к стартовому `start`.

7. Скрин с временем работы алгоритма Дейкстры

Дейкстра результат:
Длина пути (ребра на пути): 154
Время: 0.195591 секунд

8. Тесты

```

232
233 // Тестирование алгоритмов поиска пути
234 void test_pathfinding_algorithms() {
235     Graph graph;
236
237     graph.add_edge(0, 0, 1, 0, 1.0);
238     graph.add_edge(1, 0, 2, 0, 1.0);
239     graph.add_edge(2, 0, 3, 0, 1.0);
240     graph.add_edge(0, 0, 0, 1, 1.0);
241     graph.add_edge(0, 1, 1, 1, 1.0);
242     graph.add_edge(1, 1, 2, 1, 1.0);
243     graph.add_edge(2, 1, 3, 1, 1.0);
244     graph.add_edge(3, 0, 3, 1, 1.0);
245
246     vector<tuple<Node, Node, vector<Node>>> test_cases = {
247         {{0, 0}, {3, 0}}, {{0, 0}, {1, 0}, {2, 0}, {3, 0}},
248         {{0, 0}, {2, 1}}, {{0, 0}, {0, 1}, {1, 1}, {2, 1}},
249         {{1, 1}, {1, 1}}, {{1, 1}},
250         {{3, 0}, {0, 2}}, {}
251     };
252
253     for (auto& [start, end, expected_path] : test_cases) {
254         cout << "\nПример для теста (" << start.first << ", " << start.second << ") to ("
255             << end.first << ", " << end.second << ")" << endl;
256
257         auto bfs_path = bfs(graph, start, end);
258         cout << "BFS результат: ";
259         graph.print_path(bfs_path);
260
261         auto dfs_path = dfs(graph, start, end);
262         cout << "DFS результат: ";
263         graph.print_path(dfs_path);
264
265         auto dijkstra_path = dijkstra(graph, start, end);
266         cout << "Дейкстра результат: ";
267         graph.print_path(dijkstra_path);
268     }
269 }
270

```

```

270
271 int main() {
272     Graph graph;
273     graph.get_file_data("/Users/thevaldemar/Downloads/spb_graph.txt");
274     Node home = graph.find_nearest_node(30.285510, 60.028110);
275     Node itmo = graph.find_nearest_node(30.308726, 59.956435);
276
277     test_pathfinding_algorithms();
278
279     cout << "=====" << endl;
280     clock_t start_time1 = clock();
281     auto bfs_path = bfs(graph, home, itmo);
282     clock_t end_time1 = clock();
283     double duration1 = (double)(end_time1 - start_time1) / CLOCKS_PER_SEC;
284     cout << "BFS результат: " << endl;
285     cout << "Длина пути (ребра на пути): " << bfs_path.size() - 1 << endl;
286     cout << "Время: " << duration1 << " секунд" << endl;
287     graph.print_path(bfs_path);
288     cout << "=====" << endl;
289
290     clock_t start_time4 = clock();
291     auto dfs_path = dfs(graph, home, itmo);
292     clock_t end_time4 = clock();
293     double duration4 = (double)(end_time4 - start_time4) / CLOCKS_PER_SEC;
294     cout << "DFS результат: " << endl;
295     cout << "Длина пути (ребра на пути): " << dfs_path.size() - 1 << endl;
296     cout << "Время: " << duration4 << " секунд" << endl;
297     graph.print_path(dfs_path);
298     cout << "=====" << endl;
299
300     clock_t start_time2 = clock();
301     auto dijkstra_path = dijkstra(graph, home, itmo);
302     clock_t end_time2 = clock();
303     double duration2 = (double)(end_time2 - start_time2) / CLOCKS_PER_SEC;
304     cout << "Дейкстра результат: " << endl;
305     cout << "Длина пути (ребра на пути): " << dijkstra_path.size() - 1 << endl;
306     cout << "Время: " << duration2 << " секунд" << endl;
307     graph.print_path(dijkstra_path);
308     cout << "=====" << endl;
309
310     return 0;
311 }
312

```

Результаты тестов:

```

Пример для теста (0, 0) to (3, 0)
BFS результат: (0, 0) -> (1, 0) -> (2, 0) -> (3, 0)
DFS результат: (0, 0) -> (0, 1) -> (1, 1) -> (2, 1) -> (3, 1) -> (3, 0)
Дейкстра результат: (0, 0) -> (1, 0) -> (2, 0) -> (3, 0)

Пример для теста (0, 0) to (2, 1)
BFS результат: (0, 0) -> (0, 1) -> (1, 1) -> (2, 1)
DFS результат: (0, 0) -> (0, 1) -> (1, 1) -> (2, 1)
Дейкстра результат: (0, 0) -> (0, 1) -> (1, 1) -> (2, 1)

Пример для теста (1, 1) to (1, 1)
BFS результат: (1, 1)
DFS результат: (1, 1)
Дейкстра результат: (1, 1)

Пример для теста (3, 0) to (0, 2)
BFS результат: Путь не найден
DFS результат: Путь не найден
Дейкстра результат: Путь не найден

```

9. Сравнение методов и времени работы

DFS – работает через глубину, то есть он проходит сначала один путь до конца, а потом другой пути из той же точки до конца. Он будет работать медленнее всех остальных в взвешенном графе.

BFS – работает через ширину, то есть он проходит граф по уровням, находя короткий путь по итогу быстрее из 3 алгоритмов, которые я использовал.

Алгоритм Дейкстры - ищет путь по минимальному весу ребер. Он работает быстрее чем DFS, но медленнее BFS.

10. Вывод

После выполнения данной лабораторной работы могу сделать вывод, что задача поиска пути в графе была успешно решена всеми тремя алгоритмами: BFS, DFS и алгоритмом Дейкстры. По результатам тестов BFS продемонстрировал наиболее быстрое выполнение задачи в данной ситуации. DFS обработал значительное число узлов, что объясняется отсутствием ограничений на глубину поиска. В глубину искать неэффективно для поиска кратчайшего пути. Алгоритм Дейкстры обеспечил нахождение точного кратчайшего пути в условиях взвешенного графа, но его выполнение заняло больше времени по сравнению с BFS из-за необходимости использовать приоритетную очередь (ищем минимальный вес ребра)